

Machine Learning & Deep Learning

Systematic Problem-Solving Framework

👤 Structured Approach to ML/DL Solutions

May 25, 2025

🔍 Key Questions to Ask

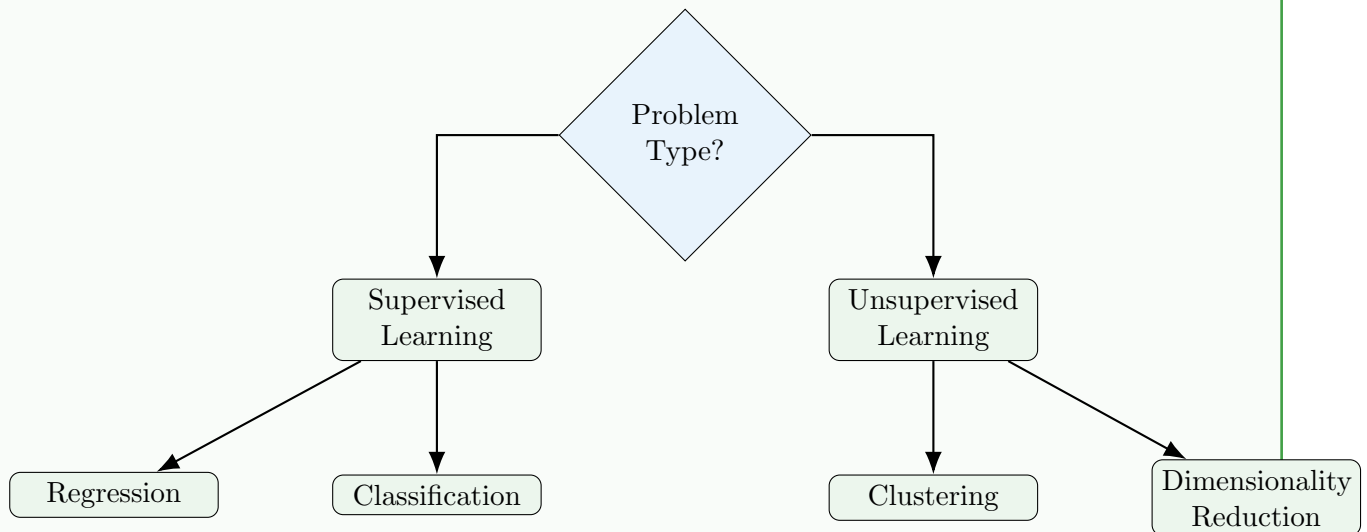
- **Problem Characteristics**

- ➤ What is the exact task? (Classification/Regression/Clustering/etc.)
- ➤ What are the inputs and expected outputs?
- ➤ What are the success criteria? (Accuracy, Latency, etc.)

- **Data Assessment**

- ➤ Data size: n samples $\times$ d features
- ➤ Data type: Tabular/Image/Text/Time-series/Graph
- ➤ Data quality: Missing values, outliers, noise
- ➤ Class distribution (for supervised learning)

💡 Problem Type Decision Tree



⚙️ Phase 2: Algorithm Selection

🔍 Algorithm Selection Framework

- **Constraints Analysis**
 - ⌚ Training time requirements
 - ⚡ Inference speed needs
 - 🏢 Hardware limitations
 - 👁 Interpretability requirements
- **Data Characteristics**
 - Small data ($n < 1k$): Linear models, k-NN
 - Medium data ($1k < n < 100k$): Tree-based, SVM
 - Large data ($n > 100k$): Deep Learning
 - High dimensions ($d > 1k$): Regularized models

💡 Algorithm Selection Matrix

Problem Type	Small Data	Medium Data	Large Data
Binary Classification	Logistic Regression, SVM	Random Forest, XGBoost	Deep NN, Ensemble
Multi-class	Naive Bayes, k-NN	Gradient Boosting	Transformer
Regression	Linear/Ridge	XGBoost, SVR	Deep NN
Image	CNN (transfer)	ResNet, EfficientNet	Vision Transformer
NLP	TF-IDF + LR	LSTM, GRU	BERT, GPT
Time Series	ARIMA, Prophet	LSTM, TCN	Transformer

🏗️ Phase 3: Model Architecture

🔍 Architecture Design Principles

- **Neural Networks**
 - Start simple (2-3 layers), then increase complexity
 - ReLU for hidden layers, appropriate output activation
 - Regularization: Dropout (0.2-0.5), BatchNorm, L2
- **Traditional ML**
 - Regularization strength (Ridge/Lasso λ)
 - Tree depth and ensemble size
 - Kernel selection for SVM

💡 Common Architectures by Task

Computer Vision:

- CNN → ResNet → EfficientNet → ViT

Natural Language:

- LSTM → Transformer → BERT/GPT

Tabular Data:

- XGBoost → TabNet → FT-Transformer

Time Series:

- LSTM → TCN → Transformer

Phase 4: Loss Function & Optimization

Loss Function Cheat Sheet

Task	Loss Function	When to Use
Binary Classification	BCE	Standard binary tasks
Multi-class	CCE	Standard classification
Imbalanced Data	Focal Loss	Class imbalance
Regression	MSE	Standard regression
Robust Regression	Huber Loss	Noisy data
Multi-label	Binary BCE	Multiple labels

Optimizer Selection Guide

- > **Adam**: Default choice (lr=3e-4)
- > **SGD+momentum**: Better generalization (lr=0.1, momentum=0.9)
- > **AdamW**: Better for transformers
- > **RMSprop**: Good for RNNs
- > Always use learning rate scheduling

Phase 5: Training Strategy

Training Configuration

- **Data Splitting**
 - Standard: 70/15/15 or 80/10/10
 - Small data: 5-10 fold CV
 - Time series: Forward chaining
- **Training Parameters**
 - Batch size: 32-256 (larger for more data)
 - Epochs: Early stopping (patience=5-10)
 - LR schedule: Cosine, ReduceOnPlateau

Phase 6: Regularization & Validation

Overfitting Red Flags

- ✗ Training loss $\ll$ Validation loss
- ✗ Training accuracy $\gg$ Validation accuracy
- ✗ Performance drops on test set

💡 Regularization Techniques

Neural Networks:

- Dropout (0.2-0.5)
- Weight decay (1e-4 to 1e-2)
- Data augmentation
- Early stopping

Traditional ML:

- L1/L2 regularization
- Feature selection
- Ensemble methods
- Cross-validation

📊 Phase 7: Evaluation & Metrics

💡 Evaluation Metrics Matrix

Task	Primary Metrics	Secondary Metrics
Binary Class	AUC-ROC, F1	Precision, Recall
Multi-class	Accuracy, Macro F1	Confusion Matrix
Imbalanced	AUC-PR, F1	Balanced Accuracy
Regression	RMSE, MAE	R^2 , MAPE
Ranking	NDCG, MAP	MRR, Hit Rate
Clustering	Silhouette	Davies-Bouldin

⚙️ Phase 8: Hyperparameter Tuning

❓ Tuning Strategy

• Key Parameters

- Learning rate (most critical)
- Model capacity (layers/units)
- Regularization strength
- Batch size

• Methods

- Grid Search: Small spaces
- Random Search: Medium spaces
- Bayesian Opt: Large spaces
- Hyperband: Limited budget

Phase 9: Deployment & Monitoring

Production Checklist

- **Model Packaging**
 - Serialization format (ONNX, PMML)
 - Version control
 - Containerization
- **Monitoring**
 - Performance metrics
 - Data drift detection
 - Logging pipeline
- **Ethics & Compliance**
 - Bias/fairness assessment
 - Privacy checks
 - Explainability

ML/DL Problem-Solving Workflow

1. Define Problem → 2. Analyze Data → 3. Select Algorithm → 4. Design Architecture
5. Choose Loss/Optimizer → 6. Train Model → 7. Evaluate → 8. Tune → 9. Deploy